

# Product and Category Management System

Relational Database Design & SQL Reference

## Project Details

**Name:** Mohanraj S

**Enrollment No.:** TU62532100211033

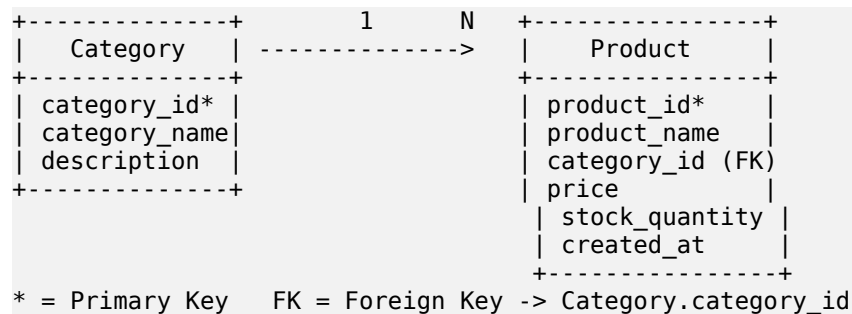
**Project Title:** Product and Category Management System

## 1. Overview

This document defines a relational database design for managing Products and Categories. It covers table structures, primary/foreign key relationships, sample data operations (insert, update, delete), and a category-wise sales/stock report query.

## 2. Entity Relationship Overview

A Category can contain many Products, but each Product belongs to exactly one Category. This is a classic one-to-many relationship, implemented using a foreign key (category\_id) in the Product table that references the primary key of the Category table.



## 3. Table Design

### 3.1 Category Table

| Column        | Data Type    | Constraint                     | Description                       |
|---------------|--------------|--------------------------------|-----------------------------------|
| category_id   | INT          | PRIMARY KEY,<br>AUTO_INCREMENT | Unique category identifier        |
| category_name | VARCHAR(100) | NOT NULL, UNIQUE               | Name of the category              |
| description   | VARCHAR(255) | NULL                           | Short description of the category |
| created_at    | TIMESTAMP    | DEFAULT<br>CURRENT_TIMESTAMP   | Record creation time              |

### 3.2 Product Table

| Column | Data Type | Constraint | Description |
|--------|-----------|------------|-------------|
|--------|-----------|------------|-------------|

|                |               |                                         |                               |
|----------------|---------------|-----------------------------------------|-------------------------------|
| product_id     | INT           | PRIMARY KEY,<br>AUTO_INCREMENT          | Unique product identifier     |
| product_name   | VARCHAR(150)  | NOT NULL                                | Name of the product           |
| category_id    | INT           | FOREIGN KEY -><br>Category(category_id) | Links product to its category |
| price          | DECIMAL(10,2) | NOT NULL, CHECK (price >= 0)            | Unit price of the product     |
| stock_quantity | INT           | NOT NULL, DEFAULT 0                     | Units available in stock      |
| created_at     | TIMESTAMP     | DEFAULT<br>CURRENT_TIMESTAMP            | Record creation time          |

## 4. SQL: Table Creation & Key Relationships

```
-- Category table
CREATE TABLE Category (
  category_id INT AUTO_INCREMENT PRIMARY KEY,
  category_name VARCHAR(100) NOT NULL UNIQUE,
  description VARCHAR(255),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Product table
CREATE TABLE Product (
  product_id INT AUTO_INCREMENT PRIMARY KEY,
  product_name VARCHAR(150) NOT NULL,
  category_id INT NOT NULL,
  price DECIMAL(10,2) NOT NULL CHECK (price >= 0),
  stock_quantity INT NOT NULL DEFAULT 0,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_product_category
    FOREIGN KEY (category_id)
    REFERENCES Category(category_id)
    ON UPDATE CASCADE
    ON DELETE RESTRICT
);
```

ON DELETE RESTRICT prevents deleting a category while products still reference it (use ON DELETE CASCADE instead if child products should be removed automatically with their category).

## 5. Product Insertion, Updating and Deletion

### 5.1 Insert Categories

```
INSERT INTO Category (category_name, description) VALUES
('Electronics', 'Electronic gadgets and accessories'),
('Groceries', 'Daily food and household essentials'),
('Furniture', 'Home and office furniture');
```

### 5.2 Insert Products

```
INSERT INTO Product (product_name, category_id, price, stock_quantity) VALUES
('Wireless Mouse', 1, 799.00, 50),
('Bluetooth Speaker', 1, 1999.00, 30),
('Basmati Rice 5kg', 2, 650.00, 100),
('Office Chair', 3, 4500.00, 15);
```

## 5.3 Update a Product

Update the price and stock of a product after a restock or price revision:

```
UPDATE Product
SET price = 849.00,
    stock_quantity = stock_quantity + 25
WHERE product_id = 1;
```

## 5.4 Delete a Product

```
DELETE FROM Product
WHERE product_id = 2;
```

To safely remove a product only when it has no stock, add a guard condition: `WHERE product_id = 2 AND stock_quantity = 0`.

## 6. Category-wise Product Report

The following query joins Product with Category to summarize product count, total stock, and inventory value per category:

```
SELECT
    c.category_name,
    COUNT(p.product_id) AS total_products,
    SUM(p.stock_quantity) AS total_stock,
    ROUND(SUM(p.price * p.stock_quantity), 2) AS inventory_value
FROM Category c
LEFT JOIN Product p
    ON c.category_id = p.category_id
GROUP BY c.category_id, c.category_name
ORDER BY inventory_value DESC;
```

### Sample Report Output

| Category    | Total Products | Total Stock | Inventory Value (INR) |
|-------------|----------------|-------------|-----------------------|
| Furniture   | 1              | 15          | 67,500.00             |
| Electronics | 2              | 80          | 119,895.00            |
| Groceries   | 1              | 100         | 65,000.00             |

### 6.1 Product Listing Within Each Category

```
SELECT
    c.category_name,
    p.product_name,
    p.price,
    p.stock_quantity
FROM Category c
JOIN Product p
    ON c.category_id = p.category_id
ORDER BY c.category_name, p.product_name;
```

## 7. Summary

- Category (1) – Product (Many) relationship enforced via a foreign key.
- Primary keys auto-generate unique IDs; the foreign key maintains referential integrity.
- INSERT / UPDATE / DELETE statements manage product lifecycle.
- JOIN + GROUP BY queries produce category-wise inventory reports.